CS1910 Computer Science I • Sequences **Partner workbook**

One-on-One Basketball

In lecture with a partner • about 30 minutes

Name: _____ **Partner:** _____

This PDF can be filled in: click in an answer space to type, or print it and write. Draw diagrams with your PDF viewer's pen or markup tool, or on paper.

What this workbook practises

A string can hold a whole record of events, one character after another. You will use computational thinking to plan a program that walks through such a string and decides who won a game.

The rules

- Games go to 11, but you must win by 2. If the game is tied 10–10, the first player to get 2 points ahead wins.
- Each shot is worth 1 or 2 points.
- A game record is one string, in the order the points were scored: the player (`A` or `B`) followed by the number of points they scored.

A2	A2	A2	A1	A2	A1	B2	B2	B2	B2	B2	B2

`"A2A2A2A1A2A1B2B2B2B2B2B2"`: player A scores the first 10 points, then player B scores the next 12 and wins.

1 Understand

about 8 minutes

1. What is the input, and what is the output?

2. Trace the example. Fill in the score after each basket.

Basket	A2	A2	A2	A1	A2	A1	B2	B2	B2	B2	B2	B2
A	2	4										
B	0	0										

3. Make up two more game records of your own: one where A wins 11–9, and one that goes to a 10–10 tie before someone wins.

2 Design

about 10 minutes

4. **Decomposition:** what smaller tasks does the program need to do?

5. **Patterns:** what repeats? How far through the string do you move each time?

6. **Abstraction:** what details can be filtered out? What do you actually need to keep track of?

7. Write the exact condition that means “someone has won”.

3 Implement and validate

about 12 minutes

8. Write `get_winner(game)`, which returns `"A"` or `"B"`.

9. **Validation:** how can you tell your program is working? List test cases and the expected winner.

Game record	Winner	Why this test?
`"A2A2A2A1A2A1B2B2B2B2B2B2"`	B	the example

Before you leave

about 1 minutes

- [ ] I can loop through a string in steps of 2 and use indexing to read each part.
- [ ] I can keep running totals instead of storing every event.
- [ ] I can write test cases, including the deuce case, before trusting my code.